

AGRIAI 2.0

INTELLIGENT FARMING ASSISTANT FOR GHANA

Volume I. Framework and Construction

How the whole AgriAI system was assembled, file by file

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Computer Science, University of Ghana

Founder, designer, and sole engineer of AgriAI

Repository: ThaddeusHackz/agriai-v2

Document date: 18 August 2026

Classification: Internal forensic architecture pack

This document explains how AgriAI was built, which source files make the AI work, and how the public and admin APIs stay reliable when a provider is missing. Every claim below is taken from the live codebase, not from marketing copy.

1. Purpose of this forensic volume

This volume is a construction record. It answers a simple question: how was AgriAI 2.0 put together so that a Ghanaian farmer can open one website and receive chat advice, crop disease analysis, weather, market prices, and voice, while an administrator can control the same product from a private panel? The answers come from the repository layout, the Next.js App Router, the data engine, and the deployment blueprint.

2. Product identity

AgriAI 2.0 is a full stack TypeScript application. Package name `agriai`, version 2.0.0. The public description in `package.json` states that it is an intelligent farming assistant for Ghana with multilingual AI chat, voice, web search, crop disease detection, market prices, weather, and a full admin panel. The founder of record in `README.md` is Thaddeus Nii Teiko Tagoe, Computer Science student at the University of Ghana, and the sole designer, developer, and maintainer.

3. Technology stack that actually ships

- Next.js 16.2.10 with the App Router and Node.js runtime on API routes.
- React 19.2.4 and React DOM 19.2.4 for the public site and admin shell.
- TypeScript 5 with a strict project `tsconfig`.

- Tailwind CSS v4 with PostCSS for the 2026 aurora and glass UI.
- Google Gemini via the official @google/genai SDK 2.16 for chat, vision, and transcription.
- Cloudflare Workers AI REST API for Llama 3.3 70B chat fallback and Flux image generation.
- OpenWeatherMap with Open-Meteo fallback for five Ghanaian cities.
- Tavily search with Wikipedia and DuckDuckGo keyless fallback.
- ElevenLabs multilingual TTS for spoken answers.
- PostgreSQL through the pg driver, plus a local atomic JSON document store.
- bcryptjs for admin password hashes and cookie sessions.
- Framer Motion, Lucide icons, react-markdown with remark-gfm, and Sonner toasts.

4. Repository map (the skeleton)

The checkout is a single Next.js project. There is no separate microservice repo. Everything a farmer or an admin needs lives under one process.

- app/page.tsx and app/layout.tsx: the public marketing and product surface.
- app/admin/page.tsx and app/admin/login/page.tsx: the control room.
- app/api/*: twenty plus HTTP route handlers. This is the entire backend.
- components/: public sections (Hero, Chat, DiseaseDetector, CropStudio, Weather, Prices) and components/admin/* tabs.
- lib/ai.ts: Gemini client, model waterfall, streaming, vision, local knowledge answers.
- lib/cloudflare.ts: Workers AI chat and Flux image calls.
- lib/search.ts: Tavily plus keyless search and citation blocks.
- lib/db.ts: seed data, atomic JSON writes, analytics, knowledge base.
- lib/pg-store.ts: PostgreSQL mirror so Render deploys do not wipe state.
- lib/auth.ts: admin cookies and bcrypt.
- lib/env.ts: one place that reads GEMINI_API_KEY and aliases.
- lib/languages.ts: English, Twi, Ga, Ewe, Hausa, French instructions.
- lib/openweather.ts and lib/images.ts: weather and Unsplash helpers.
- instrumentation.ts: boot hydration from Postgres when db.json is missing.
- render.yaml and Dockerfile: production shape on Render.

5. How Next.js makes the framework hold

AgriAI uses the App Router. A file named route.ts inside app/api/chat becomes POST /api/chat. There is no Express server. Each route sets export const runtime = "nodejs" and export const maxDuration = 60 so long Gemini or Flux calls are not killed at the default edge timeout. The browser never talks to Gemini, Cloudflare, Tavily, OpenWeather, or ElevenLabs directly. The browser talks only to AgriAI routes. That is why API keys stay on the server and why CORS and key leakage are not a farmer facing problem.

6. User interface construction

The home page is a composition of section components controlled by `settings.showSections` in the database. An admin can hide weather or studio without a redeploy. `Chat.tsx` is the Perplexity style conversation surface: Server Sent Events, markdown, stop button, source chips, language and mode selectors. `DiseaseDetector.tsx` posts a data URL to `/api/vision`. `CropStudio.tsx` posts a farm prompt to `/api/generate`. `WeatherSection.tsx` and `MarketPrices.tsx` read `/api/weather` and `/api/prices`. The admin shell is a tabbed SPA that calls `/api/admin/*` through `components/admin/api.ts`.

7. Data framework

`lib/db.ts` is a zero extra dependency document store. On first boot it seeds settings, an admin user (bcrypt hash of `ADMIN_PASSWORD`), twelve Ghana market prices, and eight knowledge base articles (maize season, cassava mosaic, cocoa price, tomato fertilizer, yam soil, fall armyworm, farm loans, maize storage). Writes go to `data/db.json` through a temp file then rename, so a crash cannot leave a half written file. When `DATABASE_URL` is set, `schedulePostgresSave` mirrors the same document into an `agriai_state` table. `instrumentation.ts` hydrates that document back if the disk file is gone after a Render redeploy. That is how a free tier filesystem that resets still keeps chats, prices, and subscribers.

8. Security and admin framework

Admin login is `POST /api/admin/login`. Failures are rate limited (five tries per fifteen minutes). Success sets a cookie session lasting seven days. Routes under `/api/admin` check that session except login itself. Passwords never sit in plaintext in the document. Public chat input is clipped to 2000 characters. Vision images are capped near 4.5 MB. Transcription audio is capped at 25 MB. Image generation only accepts farm related subjects from a `SAFE_SUBJECTS` list so the studio cannot be used as a general image toy.

9. Deployment framework

`render.yaml` describes a web service plus managed Postgres. Environment variables for Gemini, Cloudflare, OpenWeather, Tavily, ElevenLabs, and admin seed credentials are `sync false` so they are pasted in the Render dashboard, not committed. `Dockerfile` wraps the same Next.js build for container hosts. `DEPLOYMENT.md` is the operator runbook.

10. Forensic conclusion of Volume I

AgriAI is not a prompt pasted into a chatbot widget. It is a single Next.js process designed by Thaddeus Tagoe so that UI, AI, search, weather, prices, voice, persistence, and administration share one typed data model. The next volumes explain the AI chain and the HTTP contracts that make that design work under failure.